

Tipos definidos pelo programador

Os tipos de variáveis vistos até agora podem ser classificados em duas categorias:

- Tipos básicos: **char**, **int**, **float**, **double** e **void**.
- Tipos compostos homogêneos: **array**.

Dependendo da situação que desejamos modelar em nosso programa, esses tipos podem não ser suficientes. Por esse motivo, a linguagem C permite criar novos tipos de dados a partir dos tipos básicos. Para criar um novo tipo de dado, um dos seguintes comandos pode ser utilizado:

- Estruturas: comando **struct**
- Uniões: comando **union**
- Enumerações: comando **enum**
- Renomear um tipo existente: comando **typedef**

Assim, a finalidade deste capítulo é apresentar ao leitor os mecanismos envolvidos na criação de um novo tipo dentro da linguagem C e como ele pode ser utilizado dentro do programa. Ao final, o leitor será capaz de:

- Reconhecer um novo tipo de dado.
- Criar um novo tipo de dado.
- Acessar os elementos desse novo tipo de dado.
- Distinguir entre os tipos de dados possíveis.

8.1 ESTRUTURAS: STRUCT

Uma estrutura pode ser vista como um conjunto de variáveis sob o mesmo nome, e cada uma delas pode ter qualquer tipo (ou o mesmo tipo). A ideia básica por trás

da estrutura é criar apenas um tipo de dado que contenha vários membros, que nada mais são do que outras variáveis. Em outras palavras, estamos criando uma variável que contém dentro de si outras variáveis.

8.1.1 Declarando uma estrutura

A forma geral da definição de uma nova estrutura utiliza o comando **struct**:

```
struct nome_struct{
    tipo1 campo1;
    tipo2 campo2;
    ...
    tipon campoN;
};
```

A principal vantagem do uso de estruturas é que agora podemos agrupar de forma organizada vários tipos de dados diferentes dentro de uma única variável.



As estruturas podem ser declaradas em qualquer escopo do programa (global ou local).

Apesar disso, a maioria das estruturas é declarada no escopo global. Por se tratar de um novo tipo de dado, muitas vezes é interessante que todo o programa tenha acesso à estrutura. Daí a necessidade de usar o escopo global.

Na Figura 8.1, tem-se o exemplo de uma estrutura declarada para representar o cadastro de uma pessoa.

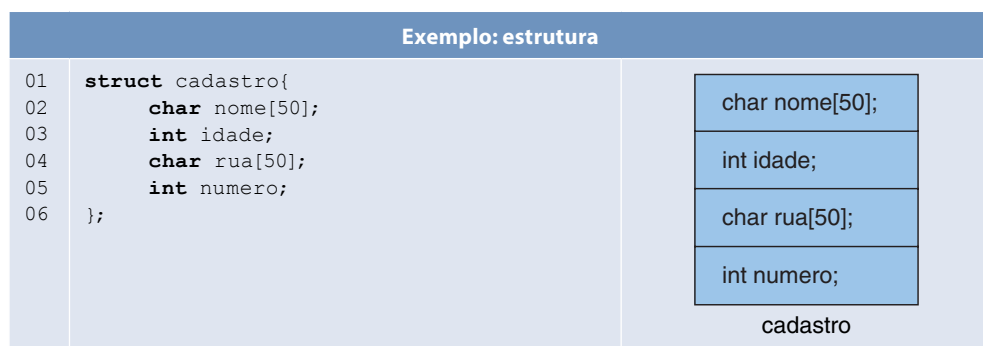


FIGURA 8.1

Note que os campos da estrutura são definidos da mesma forma que variáveis. Como na declaração de variáveis, os nomes dos membros de uma estrutura devem ser diferentes um do outro. Porém, estruturas diferentes podem ter membros com nomes iguais (Figura 8.2).

Exemplo: estrutura		
01	struct cadastro{	struct aluno{
02	char nome[50];	char nome[50];
03	int idade;	int matricula;
04	char rua[50];	float nota1,nota2,nota3;
05	int numero;	
06	};	};

FIGURA 8.2



Depois do símbolo de fechar chaves (}) da estrutura, é necessário colocar um ponto e vírgula (;).

Isso é necessário porque a estrutura pode ser também declarada no escopo local. Por questões de simplificação e por se tratar de um novo tipo, é possível, logo na definição da **struct**, definir algumas variáveis desse tipo. Para isso, basta colocar os nomes das variáveis declaradas após o comando de fechar chaves (}) da estrutura e antes do ponto e vírgula (;):

```
struct cadastro{
    char nome[50];
    int idade;
    char rua[50];
    int numero;
} cad1, cad2;
```

Nesse exemplo, duas variáveis (*cad1* e *cad2*) são declaradas junto com a definição da estrutura.

8.1.2 Declarando uma variável do tipo da estrutura

Uma vez definida a estrutura, uma variável pode ser declarada de modo similar aos tipos já existentes:

```
struct cadastro c;
```



Por ser um tipo definido pelo programador, usa-se a palavra **struct** antes do tipo da nova variável declarada.

O uso de estruturas facilita muito a vida do programador na manipulação dos dados do programa. Imagine ter de declarar quatro cadastros para quatro pessoas diferentes:

```
char nome1[50], nome2[50], nome3[50], nome4[50];
int idade1, idade2, idade3, idade4;
char rua1[50], rua2[50], rua3[50], rua4[50];
int numero1, numero2, numero3, numero4;
```

Utilizando uma estrutura, o mesmo pode ser feito da seguinte maneira:

```
struct cadastro c1, c2, c3, c4;
```

8.1.3 Acessando os campos de uma estrutura

Uma vez definida uma variável do tipo da estrutura, é preciso poder acessar seus campos (ou variáveis) para se trabalhar.



Cada campo (variável) da estrutura pode ser acessado usando o operador "." (ponto).

O operador de acesso aos campos da estrutura é o ponto (.). Ele é usado para referenciar os campos de uma estrutura. O exemplo da Figura 8.3 mostra como os campos da estrutura *cadastro*, definida anteriormente, podem ser facilmente acessados.

Exemplo: acessando as variáveis de dentro da estrutura

```

01  #include <stdio.h>
02  #include <stdlib.h>
03  #include <string.h>
04  struct cadastro{
05      char nome[50];
06      int idade;
07      char rua[50];
08      int numero;
09  };
10  int main(){
11      struct cadastro c;
12      //Atribui a string "Carlos" para o campo nome
13      strcpy(c.nome,"Carlos");
14
15      //Atribui o valor 18 para o campo idade
16      c.idade = 18;
17
18      //Atribui a string "Avenida Brasil" para o campo rua
19      strcpy(c.rua,"Avenida Brasil");
20
21      //Atribui o valor 1082 para o campo numero
22      c.numero = 1082;
23
24      system("pause");
25      return 0;
26  }
```

FIGURA 8.3

Como se pode ver, cada campo da estrutura é tratado levando em consideração o tipo que foi usado para declará-la. Como os campos *nome* e *rua* são **strings**, foi preciso usar a função **strcpy()** para copiar o valor para esses campos.



E se quiséssemos ler os valores dos campos da estrutura do teclado?

Nesse caso, basta ler cada variável da estrutura independentemente, respeitando seus tipos, como é mostrado no exemplo da Figura 8.4.

Exemplo: lendo do teclado as variáveis da estrutura

```

01  #include <stdio.h>
02  #include <stdlib.h>
03  struct cadastro{
04      char nome[50];
05      int idade;
06      char rua[50];
07      int numero;
08  };
09  int main(){
10      struct cadastro c;
11      //Lê do teclado uma string e armazena no campo nome
12      gets(c.nome);
13
14      //Lê do teclado um valor inteiro e armazena no campo idade
15      scanf("%d",&c.idade);
16
17      //Lê do teclado uma string e armazena no campo rua
18      gets(c.rua);
19
20      //Lê do teclado um valor inteiro e armazena no campo numero
21      scanf("%d",&c.numero);
22      system("pause");
23      return 0;
24  }

```

FIGURA 8.4

Note que cada variável dentro da estrutura pode ser acessada como se apenas ela existisse, não sofrendo nenhuma interferência das outras.



Lembre-se: uma estrutura pode ser vista como um simples agrupamento de dados.

Como cada campo é independente dos demais, outros operadores podem ser aplicados a cada campo. Por exemplo, pode-se comparar a idade de dois cadastros.

8.1.4 Inicialização de estruturas

Assim como nos arrays, uma estrutura também pode ser inicializada, independentemente do tipo das variáveis contidas nela. Para tanto, na declaração da variável do tipo da estrutura, basta definir uma lista de valores separados por vírgula e delimitados pelo operador de chaves ({ }).

```
struct cadastro c = { "Carlos",18,"Avenida Brasil",1082};
```

Nesse caso, como nos arrays, a ordem é mantida. Isso significa que o primeiro valor da inicialização será atribuído à primeira variável membro (*nome*) da estrutura, e assim por diante.

Elementos omitidos durante a inicialização são inicializados com 0. Se for uma string, será inicializada com uma string vazia ("").

```
struct cadastro c = { "Carlos",18 };
```

Nesse exemplo, o campo *rua* é inicializado com "" e *numero* com zero.

8.1.5 Array de estruturas

Voltemos ao problema do cadastro de pessoas. Vimos que o uso de estruturas facilita muito a vida do programador na manipulação dos dados do programa. Imagine ter de declarar quatro cadastros para quatro pessoas diferentes:

```
char nome1[50], nome2[50], nome3[50], nome4[50];
int idade1, idade2, idade3, idade4;
char rua1[50], rua2[50], rua3[50], rua4[50];
int numero1, numero2, numero3, numero4;
```

Utilizando uma estrutura, o mesmo pode ser feito da seguinte maneira:

```
struct cadastro c1, c2, c3, c4;
```

A representação desses quatro cadastros pode ser ainda mais simplificada se utilizarmos o conceito de arrays:

```
struct cadastro c[4];
```

Desse modo, cria-se um array de estruturas, em que cada posição do array é uma estrutura do tipo cadastro.



A declaração de um array de estruturas é similar à declaração de um array de um tipo básico.

A combinação de arrays e estruturas permite que se manipulem, de modo muito mais prático, diversas variáveis de estrutura. Como vimos no uso de arrays, a utilização de um índice permite que usemos o comando de repetição para executar uma mesma tarefa para diferentes posições do array. Agora, os quatro cadastros anteriores podem ser lidos com o auxílio de um comando de repetição (Figura 8.5).

Exemplo: lendo um array de estruturas do teclado

```
01 #include <stdio.h>
02 #include <stdlib.h>
03 struct cadastro{
04     char nome[50];
05     int idade;
06     char rua[50];
07     int numero;
08 };
09 int main(){
10     struct cadastro c[4];
11     int i;
12     for(i=0; i<4; i++){
13         gets(c[i].nome);
14         scanf("%d",&c[i].idade);
15         gets(c[i].rua);
16         scanf("%d",&c[i].numero);
17     }
18     system("pause");
19     return 0;
20 }
```

FIGURA 8.5



Em um array de estruturas, o operador de ponto (.) vem depois dos colchetes ([]) do índice do array.

Essa ordem deve ser respeitada, pois o índice do array é que indica qual posição do array queremos acessar, onde cada posição é uma estrutura. Somente depois de definidas as estruturas contidas no array que queremos acessar é que podemos acessar os seus campos.

8.1.6 Atribuição entre estruturas

As únicas operações possíveis em uma estrutura são as de acesso aos membros da estrutura, por meio do operador ponto (.) e as de cópia ou atribuição (=). A atribuição entre duas variáveis de estrutura faz com que os conteúdos das variáveis contidas dentro de uma estrutura sejam copiados para a outra.



Atribuições entre estruturas só podem ser feitas quando as estruturas são AS MESMAS, ou seja, quando possuem o mesmo nome!

Exemplo: atribuição entre estruturas

```

01  #include <stdio.h>
02  #include <stdlib.h>
03  struct ponto {
04      int x;
05      int y;
06  };
07
08  struct novo_ponto {
09      int x;
10      int y;
11  };
12
13  int main(){
14      struct ponto p1, p2= {1,2};
15      struct novo_ponto p3= {3,4};
16
17      p1 = p2;
18      printf("p1 = %d e %d", p1.x,p1.y);
19
20      //ERRO! TIPOS DIFERENTES
21      p1 = p3;
22      printf("p1 = %d e %d", p1.x,p1.y);
23
24      system("pause");
25      return 0;
26  }
```

FIGURA 8.6

No exemplo da Figura 8.6, *p2* é atribuído a *p1*. Essa operação está correta, pois ambas as variáveis são do tipo **ponto**. Sendo assim, o valor de *p2.x* é copiado para *p1.x* e o valor de *p2.y* é copiado para *p1.y*.

Já na segunda atribuição (*p1 = p3*;) ocorre um erro porque os tipos das estruturas das variáveis são diferentes: uma pertence ao tipo **struct ponto**, enquanto a outra

pertence ao tipo **struct** novo_ponto. Note que o mais importante é o nome do tipo da estrutura, e não as variáveis dentro dela.



No caso de trabalharmos com arrays de estruturas, a atribuição entre diferentes elementos do array também é válida.

```

01 #include <stdio.h>
02 #include <stdlib.h>
03 struct cadastro{
04     char nome[50];
05     int idade;
06     char rua[50];
07     int numero;
08 };
09 int main(){
10     struct cadastro c[10];
11     ...
12     c[1] = c[2]; //CORRETO
13
14     system("pause");
15     return 0;
16 }

```

Um array ou “vetor” é um conjunto de variáveis do mesmo tipo utilizando apenas um nome. Como todos os elementos do array são do mesmo tipo, a atribuição entre elas é possível, mesmo que o tipo do array seja uma estrutura.

8.1.7 Estruturas aninhadas

Uma estrutura pode agrupar um número arbitrário de variáveis de tipos diferentes. Uma estrutura também é um tipo de dado, com a diferença de que se trata de um tipo de dado criado pelo programador. Sendo assim, podemos declarar uma estrutura que possua uma variável do tipo de outra estrutura previamente definida. A uma estrutura que contenha outra estrutura dentro dela damos o nome **estruturas aninhadas**. O exemplo da Figura 8.7 exemplifica bem isso.

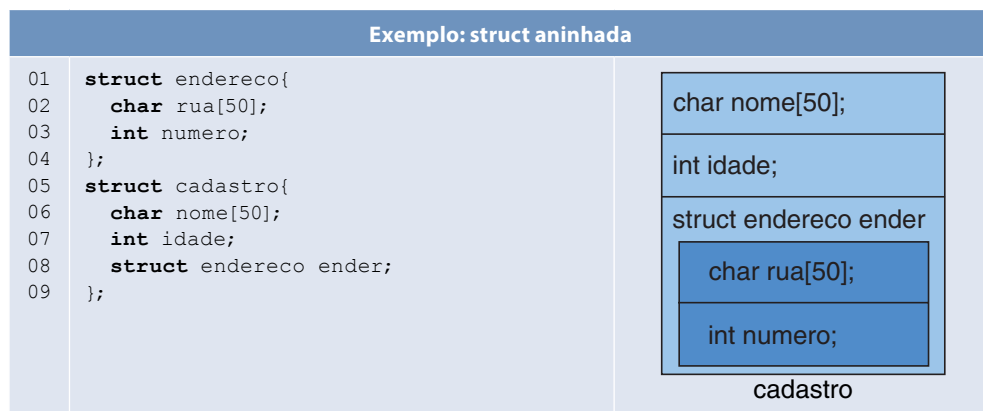


FIGURA 8.7

Nesse exemplo, temos duas estruturas: uma chamada **endereco** e outra chamada **cadastro**. Note que a estrutura **cadastro** possui uma variável *ender* do tipo **struct endereco**. Trata-se de uma estrutura aninhada dentro de outra.



No caso da estrutura **cadastro**, o acesso aos dados da variável do tipo **struct endereco** é feito utilizando-se novamente o operador “.” (ponto).

Lembre-se: cada campo (variável) da estrutura pode ser acessado usando o operador “.” (ponto). Assim, para acessar a variável *ender*, é preciso usar o operador ponto (.). No entanto, a variável *ender* também é uma estrutura. Sendo assim, o operador ponto (.) é novamente utilizado para acessar as variáveis dentro dessa estrutura. Esse processo se repete sempre que houver uma nova estrutura aninhada. O exemplo da Figura 8.8 mostra como a estrutura aninhada **cadastro** poderia ser facilmente lida do teclado.

Exemplo: lendo do teclado as variáveis da estrutura

```

01  #include <stdio.h>
02  #include <stdlib.h>
03  struct endereco{
04      char rua[50];
05      int numero;
06  };
07  struct cadastro{
08      char nome[50];
09      int idade;
10      struct endereco ender;
11  };
12  int main(){
13      struct cadastro c;
14      //Lê do teclado uma string e armazena no campo nome
15      gets(c.nome);
16
17      //Lê do teclado um valor inteiro e armazena no campo idade
18      scanf("%d",&c.idade);
19
20      //Lê do teclado uma string
21      //e armazena no campo rua da variável ender
22      gets(c.ender.rua);
23
24      //Lê do teclado um valor inteiro
25      //e armazena no campo numero da variável ender
26      scanf("%d",&c.ender.numero);
27
28      system("pause");
29      return 0;
30  }
```

FIGURA 8.8

8.2 UNIÕES: UNION

Uma união pode ser vista como uma lista de variáveis, e cada uma delas pode ter qualquer tipo. A ideia básica por trás da união é similar à da estrutura: criar apenas

um tipo de dado que contenha vários membros, que nada mais são do que outras variáveis.



Tanto a declaração quanto o acesso aos elementos de uma união são similares aos de uma estrutura.

8.2.1 Declarando uma união

A forma geral da definição de união utiliza o comando **union**:

```
union nome_union{
    tipo1 campo1;
    tipo2 campo2;
    ...
    tipon campoN;
};
```

8.2.2 Diferença entre estrutura e união

Até aqui, uma união se parece muito com uma estrutura. No entanto, diferentemente das estruturas, todos os elementos contidos na união ocupam o mesmo espaço físico na memória. Uma estrutura reserva espaço de memória para todos os seus elementos, enquanto uma **union** reserva espaço de memória para o seu maior elemento e compartilha essa memória com os demais.



Numa **struct** é alocado espaço suficiente para armazenar todos os seus elementos, enquanto numa **union** é alocado espaço para armazenar o maior dos elementos que a compõem.

Tome como exemplo a seguinte declaração de união:

```
union tipo{
    short int x;
    unsigned char c;
};
```

Essa união possui o nome **tipo** e duas variáveis: *x*, do tipo **short int** (dois bytes), e *c*, do tipo **unsigned char** (um byte). Assim, uma variável declarada desse tipo

```
union tipo t;
```

ocupará dois bytes na memória, que é o tamanho do maior dos elementos da união (**short int**).



Em uma união, apenas um membro pode ser armazenado de cada vez.

Isso acontece porque o espaço de memória é compartilhado. Portanto, é de total responsabilidade do programador saber qual dado foi mais recentemente armazenado em uma união.



Como todos os elementos de uma união se referem a um mesmo local na memória, a modificação de um dos elementos afetará o valor de todos os demais. Numa união, é impossível armazenar valores independentes.

```

01 #include <stdio.h>
02 #include <stdlib.h>
03 union tipo{
04     short int x;
05     unsigned char c;
06 };
07 int main(){
08     union tipo t;
09     t.x = 1545;
10     printf("x = %d\n",t.x);
11     printf("c = %d\n",t.c);
12     t.c = 69;
13     printf("x = %d\n",t.x);
14     printf("c = %d\n",t.c);
15     system("pause");
16     return 0;
17 }

```

Saída

```

x = 1545
c = 9
x = 1605
c = 69

```

Nesse exemplo, a variável x é do tipo **short int** e ocupa 16 bits (dois bytes) de memória. Já a variável c é do tipo **unsigned char** e ocupa os oito primeiros bits (um byte) de x . Quando atribuímos o valor 1545 à variável x , a variável c recebe a porção de x equivalente ao número 9:

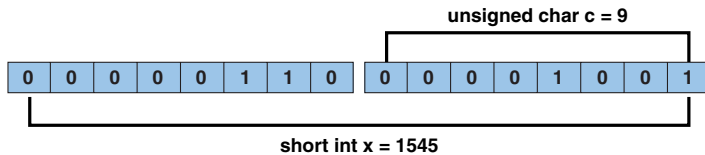


FIGURA 8.9

Do mesmo modo, se modificarmos o valor da variável c para 69, estaremos automaticamente modificando o valor da variável x para 1605:

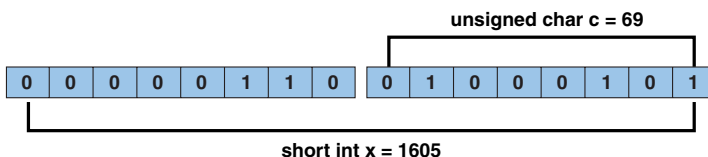


FIGURA 8.10



Um dos usos mais comuns de uma união é unir um tipo básico a um array de tipos menores.

Tome como exemplo a seguinte declaração de união:

```
union tipo{
    short int x;
    unsigned char c[2];
};
```

Sabemos que a variável *x* ocupa dois bytes na memória. Como cada posição da variável *c* ocupa apenas um byte, podemos acessar facilmente cada uma das partes da variável *x* sem precisar recorrer a operações de manipulação de bits (operações lógicas e de deslocamento de bits):

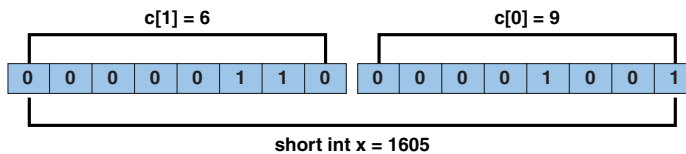


FIGURA 8.11

8.3 ENUMERAÇÕES: ENUM

Uma enumeração pode ser vista como uma lista de constantes, em que cada constante possui um nome significativo. A ideia básica por trás da enumeração é criar apenas um tipo de dado que contenha várias constantes, e uma variável desse tipo só poderá receber como valor uma dessas constantes.

8.3.1 Declarando uma enumeração

A forma geral da definição de uma enumeração utiliza o comando **enum**:

```
enum nome_enum { lista_de_identificadores };
```

Nessa declaração, **lista_de_identificadores** é uma lista de palavras separadas por vírgula e delimitadas pelo operador de chaves (`{}`). Essas palavras constituem as constantes definidas pela enumeração. Por exemplo, o comando

```
enum semana {Domingo, Segunda, Terca, Quarta, Quinta, Sexta, Sabado};
```

cria uma enumeração de nome **semana**, cujos valores constantes são os nomes dos dias da semana.



As enumerações podem ser declaradas em qualquer escopo do programa (global ou local).

Apesar disso, a maioria das enumerações é declarada no escopo global. Por se tratar de um novo tipo de dado, muitas vezes é interessante que todo o programa tenha acesso à enumeração. Daí a necessidade de usar o escopo global.



Depois do símbolo de fechar chaves (`}`) da enumeração, é necessário colocar um ponto e vírgula (`;`).

Isso é necessário porque a enumeração pode ser também declarada no escopo local. Por questões de simplificação e por se tratar de um novo tipo, é possível logo na

definição da enumeração definir algumas variáveis desse tipo. Para isso, basta colocar os nomes das variáveis declaradas após o comando de fechar chaves (}) da enumeração e antes do ponto e vírgula (;):

```
enum semana {Domingo, Segunda, Terca, Quarta, Quinta, Sexta, Sabado} s1, s2;
```

Nesse exemplo, duas variáveis (*s1* e *s2*) são declaradas junto com a definição da enumeração.

8.3.2 Declarando uma variável do tipo da enumeração

Uma vez definida a enumeração, uma variável pode ser declarada de modo similar aos tipos já existentes

```
enum semana s;
```

e inicializada como qualquer outra variável, usando, para isso, uma das constantes da enumeração

```
s = Segunda;
```



Por ser um tipo definido pelo programador, usa-se a palavra **enum** antes do tipo da nova variável declarada.

8.3.3 Enumerações e constantes

Para o programador, uma enumeração pode ser vista como uma lista de constantes, em que cada constante possui um nome significativo. Porém, para o compilador, cada uma das constantes é representada por um valor inteiro, e o valor da primeira constante da enumeração é 0. Desse modo, uma enumeração pode ser usada em qualquer expressão válida com inteiros, como mostra o exemplo da Figura 8.12.

Exemplo	
01	#include <stdio.h>
02	#include <stdlib.h>
03	enum semana {Domingo, Segunda, Terca, Quarta, Quinta,
04	Sexta, Sabado};
05	int main(){
06	enum semana s1, s2, s3;
07	s1 = Segunda;
08	s2 = Terca;
09	s3 = s1 + s2;
10	printf("Domingo = %d\n",Domingo);
11	printf("s1 = %d\n",s1);
12	printf("s2 = %d\n",s2);
13	printf("s3 = %d\n",s3);
14	system("pause");
15	return 0;
16	}
Saída	Domingo = 0 s1 = 1 s2 = 2 s3 = 3

FIGURA 8.12

Nesse exemplo, as constantes **Domingo**, **Segunda** e **Terca** possuem, respectivamente, os valores 0, 1 e 2. Como o compilador trata cada uma das constantes internamente como um valor inteiro, é possível somar as enumerações, ainda que isso não faça muito sentido.

	Na definição da enumeração, pode-se definir qual valor aquela constante possuirá.
<pre> 01 #include <stdio.h> 02 #include <stdlib.h> 03 enum semana {Domingo = 1, Segunda, Terca, Quarta=7, Quinta, 04 Sexta, Sabado}; 05 int main(){ 06 printf("Domingo = %d\n",Domingo); 07 printf("Segunda = %d\n",Segunda); 08 printf("Terca = %d\n",Terca); 09 printf("Quarta = %d\n",Quarta); 10 printf("Quinta = %d\n",Quinta); 11 printf("Sexta = %d\n",Sexta); 12 printf("Sabado = %d\n",Sabado); 13 system("pause"); 14 return 0; 15 }</pre>	
Saída	<pre> Domingo = 1 Segunda = 2 Terca = 3 Quarta = 7 Quinta = 8 Sexta = 9 Sabado = 10</pre>

Nesse exemplo, a constante **Domingo** foi inicializada com o valor 1. As constantes da enumeração que não possuem valor definido são definidas automaticamente como o valor do elemento anterior acrescido de um. Assim, **Segunda** é inicializada com 2 e **Terca** com 3. Para a constante **Quarta** foi definido o valor 7. Assim, as constantes definidas na sequência após a constante **Quarta** possuirão os valores 8, 9 e 10.

	Na definição da enumeração, pode-se também atribuir valores da tabela ASCII para as constantes.
<pre> 01 #include <stdio.h> 02 #include <stdlib.h> 03 enum escapes {retrocesso='\b', tabulacao='\t', novalinha='\n'}; 04 int main(){ 05 enum escapes e = novalinha; 06 printf("Teste %c de %c escrita\n",e,e); 07 e = tabulacao; 08 printf("Teste %c de %c escrita\n",e,e); 09 system("pause"); 10 return 0; 11 }</pre>	
Saída	<pre> Teste de Escrita Teste de escrita</pre>

8.4 COMANDO TYPEDEF

A linguagem C permite que o programador defina os seus próprios tipos com base em outros tipos de dados existentes. Para isso, utiliza-se o comando **typedef**, cuja forma geral é:

```
typedef tipo_existente novo_nome;
```

em que **tipo_existente** é um tipo básico ou definido pelo programador (por exemplo, uma **struct**) e **novo_nome** é o nome para o novo tipo que estamos definindo.



O comando **typedef** NÃO cria um novo tipo. Ele apenas permite que você defina um sinônimo para um tipo já existente.

Pegue como exemplo o seguinte comando:

```
typedef int inteiro;
```

O comando **typedef** não cria um novo tipo chamado *inteiro*. Ele apenas cria um sinônimo (*inteiro*) para o tipo **int**. Esse novo nome se torna equivalente ao tipo já existente.



No comando **typedef**, o sinônimo e o tipo existente são equivalentes.

```
01 #include <stdio.h>
02 #include <stdlib.h>
03 typedef int inteiro;
04 int main(){
05     int x = 10;
06     inteiro y = 20;
07     y = y + x;
08     printf("Soma = %d\n",y);
09     system("pause");
10     return 0;
11 }
```

Nesse exemplo, as variáveis do tipo **int** e **inteiro** são usadas de maneira conjunta. Isso ocorre porque elas são, na verdade, do mesmo tipo (**int**). O comando **typedef** apenas disse ao compilador para reconhecer **inteiro** como um outro nome para o tipo **int**.



O comando **typedef** pode ser usado para simplificar a declaração de um tipo definido pelo programador (**struct**, **union** etc.) ou de um ponteiro.

Imagine a seguinte declaração de uma **struct**:

```
struct cadastro{
    char nome[50];
    int idade;
    char rua[50];
    int numero;
};
```

Para declarar uma variável desse tipo na linguagem C, a palavra-chave **struct** é necessária. Assim, a declaração de uma variável *c* dessa estrutura seria:

```
struct cadastro c;
```



O comando **typedef** tem como objetivo atribuir nomes alternativos aos tipos já existentes, na maioria das vezes aqueles cujo padrão de declaração é pesado e potencialmente confuso.

O comando **typedef** pode ser usado para eliminar a necessidade da palavra-chave **struct** na declaração de variáveis. Por exemplo, usando o comando

```
typedef struct cadastro cad;
```

podemos agora declarar uma variável desse tipo usando apenas a palavra **cad**:

```
cad c;
```



O comando **typedef** pode ser combinado com a declaração de um tipo definido pelo programador (**struct**, **union** etc.) em uma única instrução.

Tome como exemplo a **struct** cadastro declarada anteriormente:

```
typedef struct cadastro{
    char nome[50];
    int idade;
    char rua[50];
    int numero;
} cad;
```

Note que a definição da estrutura está inserida no meio do comando **typedef**, formando, portanto, uma única instrução. Além disso, como estamos associando um novo nome à nossa **struct**, seu nome original pode ser omitido da declaração da **struct**:

```
typedef struct{
    char nome[50];
    int idade;
    char rua[50];
    int numero;
} cad;
```




O comando **typedef** deve ser usado com cuidado porque ele pode produzir declarações confusas.

```

01  #include <stdio.h>
02  #include <stdlib.h>
03  typedef unsigned int positivos[5];
04  int main(){
05      positivos v;
06      int i;
07      for (i = 0; i < 5; i++){
08          printf("Digite o valor de v[%d]:",i);
09          scanf("%d",&v[i]);
10      }
11
12      for (i = 0; i < 5; i++)
13          printf("Valor de v[%d]: %d\n",i,v[i]);
14
15      system("pause");
16      return 0;
17  }
```

Nesse exemplo, o comando **typedef** é usado para criar um sinônimo (**positivos**) para o tipo “array de cinco inteiros positivos” (**unsigned int [5]**). Apesar de válida, essa declaração é um tanto confusa, já que o novo nome (**positivos**) não dá nenhum indicativo de que a variável declarada (**v**) seja um array, nem seu tamanho.

8.5 EXERCÍCIOS

- 1) Implemente um programa que leia o nome, a idade e o endereço de uma pessoa e armazene esses dados em uma estrutura. Em seguida, imprima na tela os dados da estrutura lida.
- 2) Crie uma estrutura para representar as coordenadas de um ponto no plano (posições X e Y). Em seguida, declare e leia do teclado um ponto e exiba a distância dele até a origem das coordenadas, isto é, a posição (0,0).
- 3) Crie uma estrutura para representar as coordenadas de um ponto no plano (posições X e Y). Em seguida, declare e leia do teclado dois pontos e exiba a distância entre eles.
- 4) Crie uma estrutura chamada Retângulo. Essa estrutura deverá conter o ponto superior esquerdo e o ponto inferior direito do retângulo. Cada ponto é definido por uma estrutura Ponto, a qual contém as posições X e Y. Faça um programa que declare e leia uma estrutura Retângulo e exiba a área e o comprimento da diagonal e o perímetro desse retângulo.
- 5) Usando a estrutura Retângulo do exercício anterior, faça um programa que declare e leia uma estrutura Retângulo e um Ponto, e informe se esse ponto está ou não dentro do retângulo.

- 6) Crie uma estrutura representando um aluno de uma disciplina. Essa estrutura deve conter o número de matrícula do aluno, seu nome e as notas de três provas. Agora, escreva um programa que leia os dados de cinco alunos e os armazene nessa estrutura. Em seguida, exiba o nome e as notas do aluno que possui a maior média geral dentre os cinco.
- 7) Crie uma estrutura representando uma hora. Essa estrutura deve conter os campos hora, minuto e segundo. Agora, escreva um programa que leia um vetor de cinco posições dessa estrutura e imprima a maior hora.
- 8) Crie uma estrutura capaz de armazenar o nome e a data de nascimento de uma pessoa. Agora, escreva um programa que leia os dados de seis pessoas. Calcule e exiba os nomes da pessoa mais nova e da mais velha.
- 9) Crie uma estrutura representando um atleta. Essa estrutura deve conter o nome do atleta, seu esporte, idade e altura. Agora, escreva um programa que leia os dados de cinco atletas. Calcule e exiba os nomes do atleta mais alto e do mais velho.
- 10) Usando a estrutura “atleta” do exercício anterior, escreva um programa que leia os dados de cinco atletas e os exiba por ordem de idade, do mais velho para o mais novo.
- 11) Escreva um programa que contenha uma estrutura representando uma data válida. Essa estrutura deve conter os campos dia, mês e ano. Em seguida, leia duas datas e armazene nessa estrutura. Calcule e exiba o número de dias que decorreram entre as duas datas.
- 12) Crie uma enumeração representando os dias da semana. Agora, escreva um programa que leia um valor inteiro do teclado e exiba o dia da semana correspondente.
- 13) Crie uma enumeração representando os meses do ano. Agora, escreva um programa que leia um valor inteiro do teclado e exiba o nome do mês correspondente e quantos dias ele possui.
- 14) Crie uma enumeração representando os itens de uma lista de compras. Agora, escreva um programa que leia um valor inteiro do teclado e exiba o nome do item comprado e o seu preço.
- 15) Crie uma união contendo dois tipos básicos diferentes. Agora, escreva um programa que inicialize um dos tipos dessa união e exiba em tela o valor do outro tipo.